Lecture 05: integers in binary

Goals

- convert a number between decimal and binary unsigned integer representations
- convert a number between decimal and two's complement integer representations
- perform arithmetic on values represented as binary unsigned integers and binary two's complement
- build a circuit to do it for you
- define *overflow* and *underflow*

we've talked about logic gates, which perform operations on zeroes and ones

but this is still a far cry from letting us represent movies and music and images and books

how can we describe these really complex media such that computers can operate on them?

instead of coming up with hardware that can operate on more complex media, we're going to see how th be described using bits

but for that, we're going to need more abstractions!

these are *data* abstractions as opposed to *computation* abstractions, though

to be fair, however, the former necessitates the latter

starting easy: numbers

consider the number 763

we think of this as seven hundred sixty three

but why? what is it about those symbols that tells us the value of the whole thing?

their location!

the 7 is in the hundreds place, the 6 is in the 10s place, and the 3 is in the ones place

so we can interpret the whole value as 7 * 100 + 6 * 10 + 3 * 1

written slightly differently: 7 * 10^2 + 6 * 10^1 + 3 * 10^0

this second way to write it makes the pattern just a bit more clear and, more importantly, easier to transit

what do I mean by "base" ?

note that, in the number above, there are 10 different choices for a given digit (zero through nine) and als the second rendition is 10

these two facts are intimately related to each other, and are the *base* of this numeral system (you might a mean this, likely more on the math-y side of things)

so this is "base 10", also known as "decimal" (from Greek "deka" meaning 10)

how do we relate this back to our discussion of wires and logic and circuits?

recall that we said wires can carry a value of zero or one

so instead of 10 choices in our base-10 system, we have two choices, giving us a base-2 system

this system is usually called "binary" (again, from ancient words meaning 2)

interpreting a binary number follows the same pattern as interpreting a base-10 number, except that the be 2

thus, 110010 in binary is the value 1 * 2^5 + 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 32 + 16 + 2

in a binary number, each digit is called a *bit*, which is short for "binary digit"

so a bit is a value that can be zero or one

you can convert a binary number to its decimal equivalent using the method above

I will also expect you to be able to convert from a decimal number to its binary equivalent

the process for this is a bit more involved, but still pretty straightforward

let's work with the number 153 as an example

first, we ask if it's odd: it is, so we write down a 1 on the right-hand side of our answer

then we subtract 1 from the number we're working with and divide by 2, leaving us with 76 and an answe

now, we ask if 76 is odd; it is not, so we write a 0 to the left of our answer (giving us 01) and then divide

38 is not odd, so we add another zero to the left of our answer (now 001) and divide 36 by 2 to get 19

19 is odd, so we write a 1 to the left of our answer (now 1001), subtract 1 from 19 and divide by 2, leavin

9 is odd, so we write another 1 (11001), subtract 1 and divide by 2, leaving 4

4 is not odd, so we write a zero (011001) and divide by 2, leaving 2

2 is not odd, so we write a zero (0011001) and divide by 2, leaving 1

1 is odd, so we write a one (10011001), subtract 1, leaving a zero and we're done

so the answer is that 10011001 is the binary representation of the decimal number 153

now try this yourself: convert the number 162 to binary using the same method

(the answer is 10100010)

so now we've got a way to represent integers; what can we do with this representation?

let's try addition

first, how does addition work with decimal numbers?

thinking back to grade school, you may think about writing one number below the other and adding in co

```
  1
  153
+ 162
-----
  315
```

(where that extra '1' on the top row indicates the incoming carry from adding 5 + 6)

it turns out that we add binary in pretty much the same way, but we have to be particularly mindful of the zero or a one in each spot

so 0 + 0 gives a zero

and 1 + 0 gives a one, as does 0 + 1

but 1 + 1 cannot give us 2 because the digit 2 does not exist in binary

instead, we represent it as 10, which means "0 carry 1"

so to add together the two binary values we calculated above…

```
  1
   10011001
+  10100010
-----------
   100111011
```

(again, the top row indicates carries)

being able to perform addition on the board is all well and good, but in this class, we're interested in circu

(how) can we build a circuit to do this addition for us?

if we consider the operations we went through to perform addition on the board, there was a lot of repetit
much the same steps for each column

this suggests that, if we solve the "how to build a circuit" problem for a single column, we might be able t
other columns, too

(this is a general piece of advice for computer science: look for the commonalities, try to find a general s
many places)

so let's look at the right-most column: the input is two bits, one from each of the larger input values

and the output is also two bits: one of the sum (which we wrote below the pair of input bits) and the othe
wrote *above* the *next* pair of input bits)

since the same combination of input bits should always produce the same output bits in this case, we kn
called for

therefore we can produce a truth table:

A	B	sum	carry-out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

now, we analyze each output individually

can we produce a circuit that matches the behavior of the "sum" column? yes! it's exclusive-OR (XOR)

what about carry-out? even easier: it's just AND

so the resulting circuit looks like this: half_adder.circ

we now have a circuit that implements the behavior of the right-most column

can we use it to implement the others?

not directly, no: while the other columns also take two input bits and have two output bits, there is anothe
columns might also have a carry-in value from the preceding column

so the circuit that implements the behavior of any columns except the right-most must have three one-bit

I invite you to write down the corresponding truth table and implement the circuit yourself

the result is called a *full adder*

this is in contrast to the *half adder* from above, which only has two one-bit inputs

if we want to add together two 8-bit unsigned integers, we could take one half adder and seven full adde together

the result is called a *ripple-carry adder*

hardware (ie, circuitry) is ultimately performing operations like addition

hardware is more or less set in (proverbial) stone once it leaves the manufacturing facility

immediately relevant, the hardware that holds numbers is going to be of fixed size

therefore numbers are, effectively, a fixed number of bits

in the above addition, we can store each of the inputs in 8 bits

but the result doesn't fit: it requires 9 bits

we call this "overflow": when a value is too positive to fit in the number of bits available

it would be disingenuous to claim this scheme allows us to describe all integers: it ignores negatives!

thus, to be more precise, we call this *unsigned binary* (if you hear me say just "binary", I almost certainly you might want to ask just to be sure)

it's called "unsigned" because there is no notion of a plus sign or a minus sign: every value is implicitly n

a scheme to represent signed integers would be handy

there are many ways we could do this, but a particular method called "two's complement" is spectacularl

to get the two's complement representation of the number -x…

- take the *unsigned* representation of x
- flip all the bits
- add 1

example: what is -33 in two's complement?

- take the binary representation of 33: 100001
- flip all the bits: 011110
- add 1: 011111

except how do we know this is -33 and not 31?

in two's complement, the leading bit will always tell us whether the number is negative

a leading 1 means the number is negative

but again, how do we know this is -33 and not 31?

interpreting a two's complement number depends entirely on the number of bits available to use

I will always say how many bits the entire value takes up

for instance, I would ask: what is -33 in 8-bit two's complement?

- take the binary representation of 33: 00100001
- flip all the bits: 11011110
- add 1: 11011111

now it's unambiguous

if all negative numbers begin with a 1 in two's complement, what's the most positive number we can repr

01111111 = 127

what's the most negative number?

well, with 8 bits we can represent 2^8 = 256 different values

0-127 accounts for 128 of those, thus presumably we can represent 128 values *below* zero, so -128 is lik

check: what is -128 in 8-bit two's complement?

- take the binary representation of 128: 10000000
- flip all the bits: 01111111
- add one: 10000000

the amazing thing about two's complement is that we can use the exact same adder hardware we used f

example: 3 + (-3)

3 is 00000011

-3 is 11111101

adding them together, we get 100000000

the top bit disappears because we've only got 8 bits to store the result, giving us 00000000, which is the

and the reason we know this doesn't produce overflow is because the two inputs had different signs: the
together a positive number and a negative number will result in a number that MORE POSITIVE than the
MORE NEGATIVE than the original negative number

how can we perform subtraction?

easy: just add the inverse

so we calculate 19 - 43 by instead calculating 19 + (-43)

two's complement presents a condition analogous to overflow, in which the result of an arithmetic operat
represented by the number of bits available

we call this *underflow*

to slightly rephrase the note above about how we can detect overflow: underflow can only happen when
negative

Definitions

The following definitions introduced in this lecture are fair-game for future quizzes. You will be expected f
provided in these lecture notes.

- overflow
- underflow

Mechanical Skills

The following mechanical skills introduced in this lecture are fair-game for future quizzes. You may acces
exactly resemble the questions on the quizzes) on `weathertop`.

```
t1p1m03    Convert between decimal and unsigned binary
t1p1m04    Convert between decimal and two's complement
```

Last modified: 09/23/2026 22:35:51